Software Design Specification

Personal Budget Management App

CS251 – Introduction to Software Engineering

Cairo University · Faculty of Computers & Artificial Intelligence

Version 1.0 | May 2026

Student Name	Student ID	Role
Ibrahem Ahmed Ezzat	20240004	Team Lead / Backend
Mohamed Abdullah	20240515	UI Developer
Hesham Mohamed Fathy	20240649	Data Layer / Testing
Youssef Sayed Ali	20240692	Documentation / Diagrams

1. Introduction

This Software Design Specification (SDS) describes the complete architectural and detailed design of the Personal Budget Management App, a desktop application developed in Java with a JavaFX graphical interface. The document covers the system architecture, component design, class structures, data storage design, and the interactions between all system layers.

The application follows the Model-View-Controller (MVC) architectural pattern and applies SOLID design principles to ensure a maintainable, testable, and extensible codebase.

2. System Overview

The Personal Budget Management App is a Java-based desktop application that allows users to manage their personal finances. Key capabilities include:

Authentication	Secure user registration and login
Income & Transactions	Log financial transactions by category
Budget Management	Set monthly spending limits per category with threshold alerts
Savings Goals	Create financial goals and track contribution progress
Financial Reports	Analyse spending breakdown by category
JSON Persistence	All data saved to local .txt files (users, incomes, budgets)

3. Architectural Design

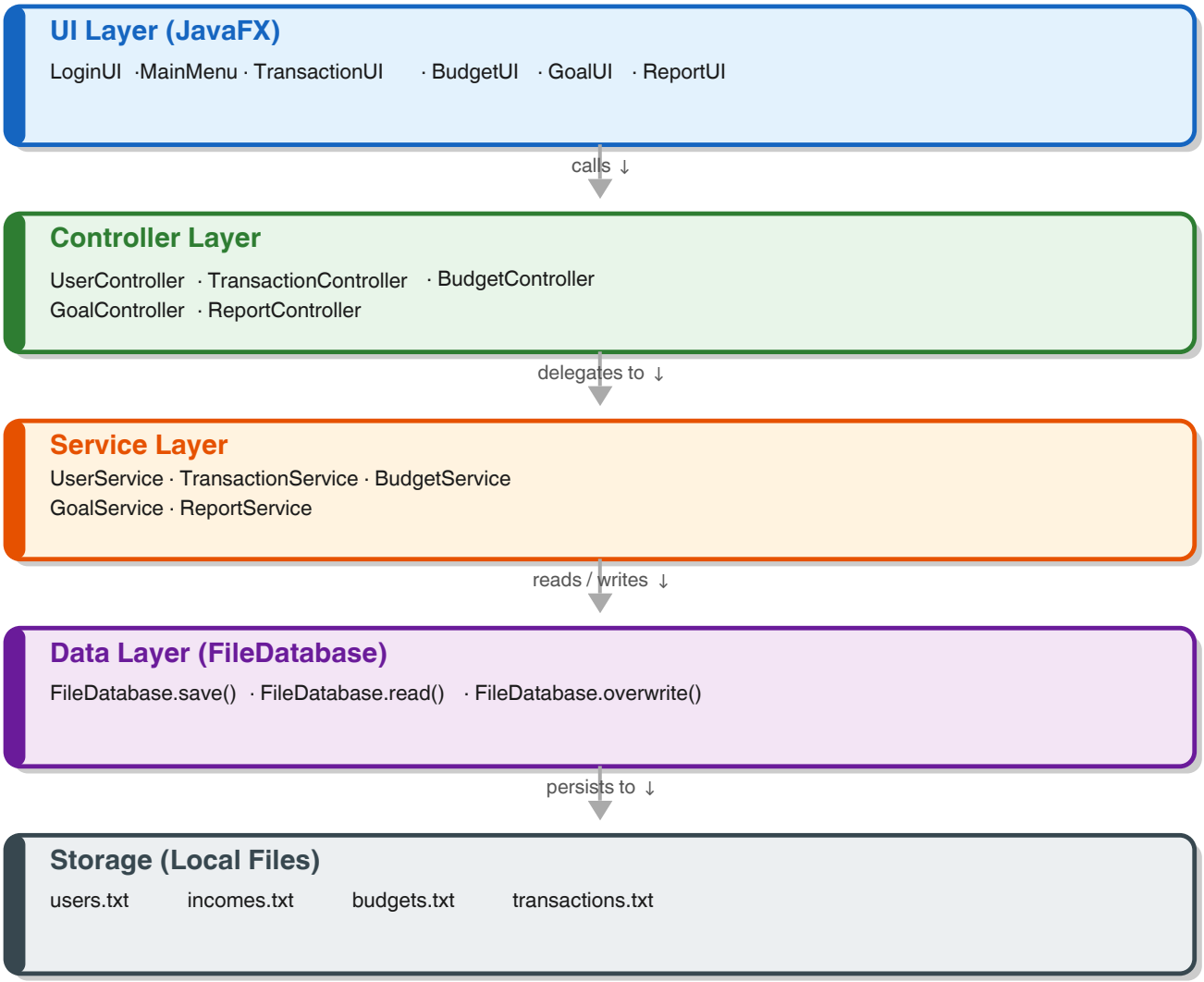
3.1 Architecture Pattern: MVC

The system is divided into four distinct layers, each with a single well-defined responsibility:

Layer	Package	Responsibility	Key Classes
Model	app.models	Plain data objects (POJOs)	User, Transaction, Budget, Goal, Income
View	app.ui	JavaFX UI screens	LoginUI, MainMenu, BudgetUI, GoalUI
Controller	app.controllers	Bridge between UI and Service	UserController, BudgetController
Service	app.services	Business logic & rules	UserService, BudgetService,Goal Service
Data	app.data	File I/O — JSON/CSV	persistencFeileDatabase

3.2 Layered Architecture Flow

The diagram below illustrates the 4-tier MVC flow from the UI through controllers and services down to persistent storage.



4. Component Design

4.1 Model Layer — app.models

Class	Attributes	Key Methods
User	id:int, name:String, password:String	legetName(), getSalary(), getPassword()
Transaction	userId:int, amount:double,category:String	getUserId(), getAmount(), getCategory()
Budget	userId:int, category:String, limit:double,	spgeentUt:dsoeurlbdl(e), getCategory(), getLimit()
Goal	setSpent()	getUserId(), getAmount(), setAmount()
Income	userId:int, name:String, target:double	getName(), getTarget()

4.2 Controller Layer — app.controllers

Controllers actas thebridgebetweenUI screensand service classes. Each controller is responsible for exactly one domain (Single Responsibility Principle).

Controller	Delegates To	Methods
UserController	UserService	signup(username, password) login(username, password) → User updateBudget(userId, category, amount)
TransactionController	TransactionService	add(userId, amount, category)
BudgetController	BudgetService	create(userId, category, limit) checkBudget(userId, category, amount)
GoalController	GoalService	create(userId, name, target) addProgress(userId, name, amount)
ReportController	ReportService	show(userId) getSummary(userId)

4.3 Service Layer — app.services

Service	Responsibility	Key Logic
UserService	Authentication & user	login(): reads file, matches credentials -> User update(): updates budget after transaction
TransactionService	Transaction recording	add(): validates amount > 0 Updates budget via BudgetService Persists transaction to file
BudgetService	Budget CRUD & spending aclreeratste()	update(): adds spending, calculates % Alerts at 80% (warning) and 100% (over limit)
GoalService	Savings goals	create(): saves goal with saved=0 contribute(): adds amount, calculates progress % Prints 'Goal Achieved!' when 100% reached
ReportService	Financial analytics	show(): reads all transactions, calculates total spending + breakdown by category summary() calls show() internally

4.4 Data Layer — app.data.FileDatabase

FileDatabase is a static utility class that abstracts all file I/O. It provides three core operations used by every service:

Method	Signature	Description
save()	save(String file, String data)	Appends a new record line to the specified file
read()	read(String file) → List<String[]>	Reads all records from file, splits each line by comma
overwrite()	overwrite(String file, List<String> lines)	Replaces entire file content (used for updates to existing

5. UI Design — app.ui

Screen Class	Constructor Args	Responsibility
LoginUI	Stage stage	Login and signup forms. On success -> navigates to MainMenu
MainMenu	Stage stage, User user	Central navigation hub — launches all other screens via sidebar
TransactionUI	Stage stage, User user	Add transaction (amount + category). Validates input, calls Trans
BudgetUI	Stage stage, User user, MainMenu Budget	CareinaMtee bnudget (category + limit). Validates and saves via
GoalUI	Stage stage, User user	Create savings goal (name + target). Validates and saves via Goal
ReportUI	Stage stage, User user	Display full financial report: total spending + breakdown by catego

6. Data Storage Design

The application uses flat-file storage. All data files are located in the data/ directory at the project root. Each line represents one record, fields separated by commas.

File	Format	Fields	Example Record
users.txt	CSV line	userId, username, password	9467,Ibrahem Ahmed,himaGo
incomes.txt	CSV line	userId, amount	9467,15000.0
budgets.txt	CSV line	userId, category, limit, spent	9467,Food,2000.0,0.0
transactions.txt	CSV line	userId, amount, category	9467,150.0,Food
goals.txt	CSV line	userId, name, target, saved	9467,Vacation,5000.0,800.0

7. SOLID Principles Applied

Principle	Application in This Project
S — Single Responsibility	Each class has one job: FileDatabase handles only I/O; each Service handles only its domain
O — Open / Closed	transaction categories can be added without changing TransactionService. New repo All UI screens
L — Liskov Substitution	implement the same show() method pattern, making them interchangeable f No fat interfaces —
I — Interface Segregation	controllers expose only the methods their specific UI screens need.
D — Dependency Inversion	UI screens depend on Controllers (abstraction layer), not directly on Services or FileDatab